

DDA Exercise 11 Study Notes

DDA Exercise 11 — Distributed Computing III: Study Notes & Code Review

Covers: (1) key concepts from the Hortonworks *Spark QuickStart Guide* PDF, (2) the theory behind this exercise, and (3) a review of your actual `app.py` / `spark.sh` / `main.sh` / `post_process.py` files, including one real bug to fix.

Part 1 — Spark QuickStart Guide (PDF): What's Actually Relevant

The PDF is from 2015 and describes Spark 1.3.1 on Hortonworks HDP/YARN — your exercise runs modern Spark (3.5.x) in **standalone mode** on Slurm, not YARN, so most installation/Ambari/Kerberos content doesn't apply directly. The transferable concepts are:

1.1 What Spark Pi actually does

The PDF's own validation example is the same Monte Carlo Pi idea you're implementing: generate random points in the unit square, count how many land inside the unit circle, and use the ratio to approximate π . This confirms the math your `is_point_inside_unit_circle` function relies on.

1.2 Deployment modes (still conceptually relevant)

- **Cluster mode:** driver runs on a cluster-managed process — good for long-running production jobs, no need to keep the submitting client alive.
- **Client mode:** driver runs in the process that submitted the job — better for interactive/debugging use, and it's what `spark.sh` uses (`-deploy-mode client`), since you want to keep the batch job around to wait for completion.

1.3 Resource configuration knobs

The PDF spells out the ones that actually matter for your scale-up/scale-out study: `--executor-cores` / `spark.executor.cores` — cores per executor (parallelism *within* a node) `--num-executors` — how many executor processes (parallelism *across* nodes/workers) `--executor-memory` / `--driver-memory` — JVM heap sizing - General guidance: don't over-allocate executors beyond what you need, and there's a tradeoff between many small executors vs. few large ones due to JVM garbage collection overhead.

These map directly onto your `spark.sh`, where `SPARK_WORKER_CORES/SPARK_EXECUTOR_CORES` and `spark.default.parallelism` are the levers you're already using for the scale-up axis.

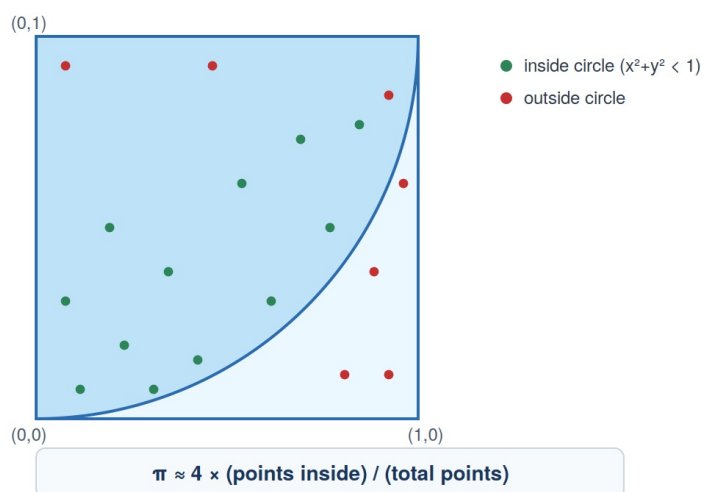
1.4 Why this PDF is otherwise mostly background reading

Ambari installation wizards, Kerberos keytabs, ORC file access, and Hive integration are irrelevant to a Slurm/standalone Spark job doing an in-memory RDD computation. Treat Part 1 of the PDF (Intro) and the Pi/resource-config sections as the useful bits; skip the rest unless you're curious about Spark's history.

Part 2 — Core Exercise Concepts

2.1 Monte Carlo estimation of π

- Inscribe a circle of radius r in a square of side $2r \rightarrow$ area ratio = $\pi/4$.
- Sample random points (x, y) in $[0,1] \times [0,1]$; a point is "inside" if $x^2 + y^2 < 1$.
- $\pi \approx 4 \times (\text{points inside}) / (\text{total points})$. Accuracy improves with more points (Law of Large Numbers).



Monte Carlo Pi geometry

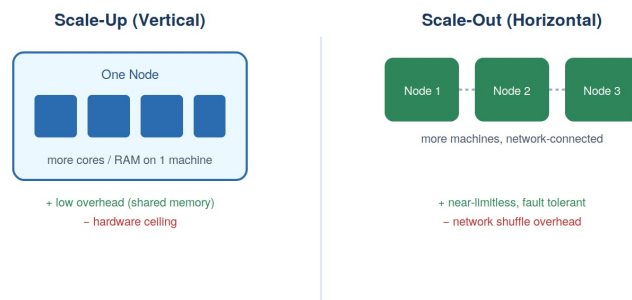
2.2 Spark execution model

- **RDD**: immutable, partitioned collection distributed across the cluster.
- **Transformations** (map, filter) are lazy — they just get recorded into a DAG (Directed Acyclic Graph), not executed. This is why `mapTime` in your results will look artificially tiny.
- **Actions** (reduce, count, collect) trigger actual execution — this is why `reduceTime` captures the *entire* pipeline (data generation + map + reduce), not just the reduce step.

2.3 Scale-up vs. scale-out

	Scale-up (vertical)	Scale-out (horizontal)
What changes	Cores/RAM on one node	Number of nodes
Overhead	Low (shared memory)	Higher (network shuffle)
Ceiling	Bounded by single-machine hardware	Theoretically open-ended

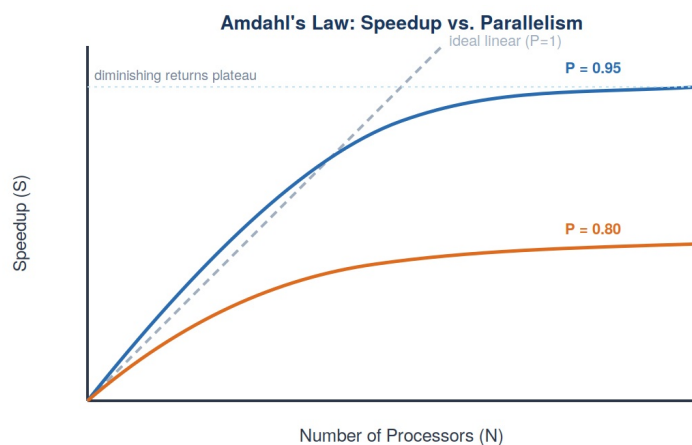
Failure mode	Single point of failure	More fault-tolerant
--------------	-------------------------	---------------------



Scale-up vs scale-out

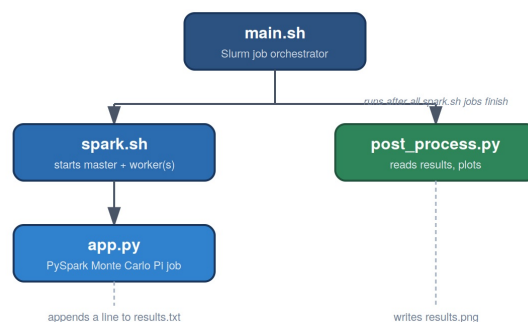
2.4 Speedup and Amdahl's Law

$S = T_1 / T_p$ $S_{max} = 1 / ((1-P) + P/N)$ Where P is the parallelizable fraction of the work and N the number of processors. Expect your speedup-vs-parallelism curve to flatten out — that plateau is Amdahl's Law showing up empirically, not a bug in your measurement.



Amdahl's Law speedup curve

2.5 Script pipeline



Script pipeline

```

main.sh (submits Slurm batch jobs, loops over node/CPU configs,
chains a post-processing job)
  |> spark.sh (starts Spark master+worker on allocated nodes,
spark-submits app.py)
    |> app.py (the actual PySpark Monte Carlo job; writes
one CSV-ish line per run)
  
```

↳ post_process.py (reads results.txt, computes speedup, plots)

Part 3 — Review of Your Actual Files

Good news: most of the marked # Task sections in your uploads already look **filled in correctly**. Here's a file-by-file check.

app.py — ✓ looks complete

- functionToMeasure times map and reduce separately with time.time() — correct.
- SparkSession.builder.appName("MonteCarloPi_HPC").getOrCreate() — correctly initializes with just an app name (no explicit master string), which is right for spark-submit against a standalone cluster.
- sc.stop(); spark.stop() at the end — correct cleanup.
- One thing to double check: total_darts = sys.argv[2] arrives as a **string**, and it's only used inside f-strings / passed to range(), so if you ever cast it (e.g., int(total_darts)), do it explicitly — range() will error on a string.

spark.sh — ✓ looks complete

- SPARK_WORKER_CORES / SPARK_EXECUTOR_CORES are set via sed from \$NUM_CPUS.
- spark-submit passes --conf spark.executor.cores=\$NUM_CPUS --conf spark.default.parallelism=\$NUM_CPUS — this is exactly the “add executor cores and parallelism information” task.
- Minor note: spark.standalone.submit.waitForAppCompletion=true isn't a real Spark standalone-mode config key (it's more associated with Mesos); it's harmless (Spark ignores unknown confs) but won't do anything — the actual blocking behavior comes from spark-submit running in the foreground in client mode, which is already correct.

post_process.py — ⚠ works, but only handles one axis

- Speedup calculation and the dual-axis plot (processing time + speedup vs. the x-variable) are implemented.
- However, it **always groups by nodes**:

```
df = df_init.groupby('nodes').mean().reset_index()
```

This is fine for **scale-out** experiments (where nodes vary and cpus is fixed), but for **scale-up** experiments nodes is fixed at 1 and cpus is the variable — grouping by nodes there would collapse all your scale-up runs into a single row and hide the effect you're trying to plot. **Fix:** parameterize which column to group by, e.g. pass it as a 3rd CLI arg or detect automatically:

```
group_col = 'cpus' if df_init['nodes'].nunique() == 1 else
'nodes'
df = df_init.groupby(group_col).mean().reset_index()
base_time = df[df[group_col] == df[group_col].min()]
['time'].values[0]
df['speedup'] = base_time / df['time']
ax1.plot(df[group_col], df['time'], ...)
ax1.set_xlabel(f'Number of {"CPUs" if group_col=="cpus" else
"Nodes"}')
```

main.sh — 🐛 real bug found

Line 151 calls:

```
python Solution_post_process.py "$OUTPUT_DIR/results.txt"
"$OUTPUT_DIR"
```

But the file you actually have is named **post_process.py**, not `Solution_post_process.py`. As written, this job will fail with a `FileNotFoundError` (no such file) once it reaches the post-processing step. Fix:

```
python post_process.py \"$OUTPUT_DIR/results.txt\" \"$OUTPUT_DIR\"
```

Also worth checking: this line doesn't `cd` into the directory containing `post_process.py` before running it, so it depends on Slurm's working directory matching where you submitted `main.sh` from. Since `main.sh` computes `OUTPUT_DIR` via `realpath $(dirname $0)`, you likely want the same pattern for locating `post_process.py`, e.g.:

```
python \"$(dirname $0)/post_process.py\" \"$OUTPUT_DIR/results.txt\"
\"$OUTPUT_DIR\"
```

Running it (./main.sh -h)

-a <account>	ZIH account/project ID
-n <nodes> (scale-out axis)	comma-separated node counts, e.g. -n 1,2,4
-c <cpus> (scale-up axis)	comma-separated cpu counts, e.g. -c 1,2,4,8
-r <runs>	repetitions per configuration
-o <output_dir>	where results/plots land

For **scale-up**, vary `-c` and keep `-n 1`. For **scale-out**, fix `-c` at whatever "best" value you found from scale-up, and vary `-n`.

Summary of Action Items

1. Fix the filename mismatch in `main.sh` (`Solution_post_process.py` → `post_process.py`), and make the path robust to working directory.
2. Update `post_process.py` to group by the varying dimension (`cpus` for scale-up, `nodes` for scale-out) instead of hardcoding `nodes`.
3. Run scale-up first (`-n 1 -c 1,2,4,8,...`) to find your best CPU count, then use that fixed value for scale-out (`-c <best> -n 1,2,4,...`).
4. Everything else in `app.py` and `spark.sh` is already correctly filled in.